REDES NEURONALES 2026

Trabajo Práctico 2: Clasificación de Fashion-MNIST con PyTorch

Objetivo general. El objetivo del trabajo es elaborar un **informe técnico breve y autocontenido** sobre el problema de clasificación de imágenes, del conjunto Fashion-MNIST, utilizando como referencia base la notebook correspondiente a la Guía de ejercicios N.º 11. El informe debe mostrar que el grupo de trabajo comprende el flujo completo del experimento: preparación de datos, definición de la red, función de pérdida, entrenamiento, validación, selección de configuración, evaluación final sobre test y análisis de errores.

Indicaciones para la entrega.

1. El trabajo práctico debe enviarse por correo electrónico a las siguientes direcciones:
   - `francisco.tamarit@unc.edu.ar`
   - `juan.perotti@unc.edu.ar`
   - `tristan.osan@unc.edu.ar`
   - `francomilana@unc.edu.ar`

   y también a las direcciones de correo electrónico de **todos los integrantes del grupo**. Es importante que tanto los docentes como todos los miembros del grupo reciban el correo de entrega.
   El asunto del correo debe ser:
   `Redes Neuronales 2026 - entrega TP2`
2. Solo se aceptan trabajos prácticos en formato `.pdf`. Opcionalmente, puede adjuntarse al correo la notebook de Google Colab utilizada para realizar los experimentos.
3. El **nombre del archivo `.pdf`** debe seguir la siguiente estructura:
   `grupo-numero_apellido1_apellido2_apellido3_TP2_RN_2026.pdf`
4. El informe completo no debe exceder las **cuatro (4) páginas**, incluyendo figuras, tablas y referencias bibliográficas. Se valorará la capacidad de síntesis y de selección de los resultados relevantes.

Formato y estructura del informe. Utilice como referencia la notebook correspondiente a la Guía de ejercicios N.º 11. El informe **no debe reproducir la notebook**; debe presentar una síntesis razonada del problema, la metodología, los resultados y su interpretación.

1. **Título.** Escriba un título claro y conciso que represente el contenido del trabajo. Por ejemplo: *"Clasificación de imágenes Fashion-MNIST con una red neuronal feedforward multicapa en PyTorch"*.
2. **Autores.** Indique el nombre completo de todos los integrantes del grupo.
3. **Resumen.** Redacte un único párrafo breve, de aproximadamente 5–8 líneas, que sintetice:
   - el problema estudiado;
   - el tipo de modelo utilizado;
   - la metodología de comparación y selección;
   - el principal resultado obtenido sobre el conjunto *test*.

   No incluya citas bibliográficas ni detalles de implementación en el resumen.
4. **Introducción.** Presente brevemente el problema de clasificación multiclase y el objetivo del trabajo. Explique, en pocos párrafos, por qué una red neuronal *feedforward* multicapa puede utilizarse para clasificar imágenes representadas como vectores. Incluya citas bibliográficas pertinentes sobre redes neuronales y/o Fashion-MNIST [1, 2].
   La introducción debe aportar contexto; **no** se espera una revisión teórica extensa de aprendizaje profundo.
5. **Fundamentos y metodología.** Describa de manera concisa el experimento efectivamente realizado. Deben quedar claros, como mínimo, los siguientes puntos:

- **Datos:** Fashion-MNIST, estructura de las imágenes y sus clases. Indique las particiones de datos utilizadas.
- **Preprocesamiento:** transformación a tensor y normalización de las entradas. Explique brevemente el propósito de esta normalización.
- **Arquitectura:** red MLP completamente conectada

$$784 \longrightarrow \cdots \longrightarrow \cdots \longrightarrow 10,$$

con activaciones ReLU y Dropout en las capas ocultas. La salida debe interpretarse como un vector de 10 *logits*.
- **Función de pérdida:** explique la relación conceptual

$$\text{logits} \longrightarrow \text{CrossEntropyLoss} \longrightarrow \text{pérdida},$$

y por qué no es necesario agregar una capa *Softmax* explícita antes de `CrossEntropyLoss`.
- **Entrenamiento:** indique cómo se entrena el modelo, los parámetros que se actualizan mediante *backpropagation* y el optimizador correspondiente.
- **Optimizadores:** compare SGD y Adam, indicando los *learning rates* utilizados. Mantenga constantes, para una comparación razonable, la arquitectura, el tamaño de *mini-batch*, *Dropout* y el número de épocas. PyTorch puede citarse como herramienta de implementación [3].
- **Evaluación:** explique claramente el rol diferenciado de *train*, *validation* y *test*.

No es necesario describir línea por línea las funciones de entrenamiento y evaluación. En particular, no se espera una explicación detallada de comandos como `train_one_epoch()`, `evaluate()`, `model.eval()` o `torch.no_grad()`. Sí debe quedar claro **qué función cumplen dentro del procedimiento experimental**.

6. **Resultados.** Presente únicamente los resultados necesarios para justificar la selección del modelo y evaluar su desempeño. Como mínimo, el informe debe contener:
   a) **Curvas de entrenamiento del modelo seleccionado:** pérdida (*loss*) y *accuracy* para *train* y *validation* en función del número de épocas. Se recomienda reunirlas en una sola figura con dos paneles para ahorrar espacio.
   b) **Comparación SGD–Adam usando validation:** una tabla breve que incluya, al menos, optimizador, *learning rate*, *validation loss* y *validation accuracy* **evaluadas al final de la última época** (es decir, para el mismo modelo que luego se evalúa sobre *test*).
   c) **Evaluación final sobre *test*:** informe *test loss* y *test accuracy* de la configuración seleccionada.
   d) **Matriz de confusión normalizada sobre el conjunto *test*:** utilice filas para las clases verdaderas y columnas para las clases predichas, normalizando por filas. Interprete la diagonal y señale las confusiones más relevantes entre clases.

   Para toda figura o tabla:
   - incluya una leyenda informativa;
   - menciónela explícitamente en el texto;
   - identifique correctamente ejes, filas y columnas;
   - no se limite a describir qué aparece: **interprete el resultado**.

   La *accuracy* por clase puede discutirse a partir de la diagonal de la matriz de confusión normalizada. No es obligatorio incluir una figura adicional si esa información ya está representada claramente.
7. **Discusión y conclusiones.** Analice los resultados y responda, en forma integrada, preguntas como las siguientes:
   - ¿qué optimizador produjo el mejor comportamiento sobre *validation* y por qué se eligió esa configuración?;
   - ¿en qué medida la conclusión sobre qué optimizador es "mejor" depende de los *learning rates* elegidos y del número fijo de épocas? ¿Había convergido SGD al final del entrenamiento?;

- ¿son las diferencias observadas entre configuraciones mayores que la variabilidad esperable entre corridas con distintas semillas?;
- ¿las curvas sugieren convergencia, subajuste o sobreajuste?;
- ¿qué clases son mejor reconocidas y cuáles presentan mayores dificultades?;
- ¿qué pares de clases se confunden con mayor frecuencia y resulta razonable esa confusión desde el punto de vista visual?;
- ¿qué modificaciones podrían explorarse para mejorar el desempeño?

Las conclusiones deben estar sustentadas por los resultados presentados. Evite afirmaciones generales que no puedan relacionarse con una figura, tabla o métrica del experimento.

8. **Referencias bibliográficas.** El informe debe incluir **al menos tres referencias relevantes**, citadas explícitamente dentro del texto. Se recomienda que al menos dos sean fuentes académicas (libros, artículos o trabajos científicos). La documentación oficial de PyTorch puede utilizarse como referencia técnica adicional, pero no debería ser la única fuente conceptual.
   Utilice un formato bibliográfico coherente. Como punto de partida pueden consultarse las fuentes sugeridas al final de este enunciado, pero pueden utilizarse otras referencias pertinentes.

Qué se espera encontrar en el informe.

- una explicación breve pero correcta del problema y de la arquitectura utilizada;
- una descripción reproducible de las decisiones experimentales principales;
- una separación metodológica clara entre *train*, *validation* y *test*;
- resultados cuantitativos suficientes para justificar la selección del modelo;
- una matriz de confusión normalizada correctamente interpretada;
- discusión de los errores, no solamente de la *accuracy* global;
- citas bibliográficas pertinentes integradas al texto;
- conclusiones vinculadas con los resultados efectivamente obtenidos.

Qué se recomienda no incluir. Para aprovechar adecuadamente el **límite de cuatro páginas**, se recomienda **no incluir**, salvo que sea indispensable para sostener un argumento:

- listados extensos de código Python o capturas de celdas de Google Colab;
- una explicación línea por línea de la implementación en PyTorch;
- mosaicos exploratorios de Fashion-MNIST ya analizados en actividades previas;
- derivaciones matemáticas extensas de *backpropagation*, *Softmax* o *Cross-Entropy Loss*;
- definiciones generales de redes neuronales que no estén conectadas con el experimento realizado;
- todas las pruebas o ejecuciones realizadas durante el desarrollo: seleccione únicamente las que sean necesarias para el argumento del informe;
- múltiples figuras que transmitan esencialmente la misma información;
- ejemplos individuales de predicciones correctas si no aportan información adicional al análisis de errores;
- resultados detallados de experimentos utilizados para decidir hiperparámetros o para elegir el optimizador.

Distribución orientativa del espacio. La siguiente distribución es solamente una recomendación para ayudar a producir un informe compacto:

Contenido	Extensión orientativa
Resumen + Introducción	0.5 página
Fundamentos y metodología	1.0 página
Resultados	1.25 páginas
Discusión y conclusiones	0.75 página
Referencias	0.5 página

Esta distribución no es obligatoria. El criterio principal es que el informe presente un argumento claro y completo dentro del máximo de cuatro páginas.

Fuentes sugeridas. Las siguientes referencias constituyen ejemplos adecuados para comenzar la búsqueda bibliográfica del informe:

Referencias

[1] I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*. MIT Press, 2016. Disponible en https://www.deeplearningbook.org/.

[2] H. Xiao, K. Rasul y R. Vollgraf, "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms", *arXiv preprint arXiv:1708.07747*, 2017.

[3] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library", *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever y R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting", *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.